演習05：ミッション・スコア・記録保存

担当案：メンバーC / 氏名は話し合いで記入

到達目標

脱出結果を数値で振り返れるように、ミッションと得点内訳を表示し、上位5件を次回起動時にも残します。

既存の制限時間と結果画面は共通ベースです。新規部分はミッション判定、内訳付き得点、JSON保存・読み込みです。先に00_commonの6〜9節を読みます。

必須：ミッション2種類、得点計算、トップ5保存、破損時の復帰。任意：ステージ別ランキング、名前入力。

練習1：ゲームから受け取る情報を決める

MissionTrackerをmissions.pyへ作ります。初期状態はdamage_count=0、key_ids=set()、result=Noneです。新しいプレイでは新しいインスタンスを作ります。

on_damageは実際にHPが減ったときだけ呼びます。敵と接触していても無敵時間中なら増やしません。on_key(key_id)はsetへ入れ、同じ鍵を2回通知しても数が増えないようにします。

finishの引数は won: bool, time_left: float, hp: int, stage_id: str。Resultの項目はcontracts.pyを参照します。

練習2：ミッションを判定する

ミッションは「脱出成功かつ被弾0回」「脱出成功かつ残り60秒以上」の2種類です。失敗したプレイで達成扱いにしません。

```
no_damage = won and self.damage_count == 0
fast_escape = won and time_left >= 60.0
```

被弾後に回復してHPが満タンになっても、ノーダメージ扱いにはなりません。最終HPだけで判定しないでください。

ミッションの成否を表示用の文字列へ直す処理と、判定そのものを分けます。まずassertで判定を確かめ、その後に画面表示を作ります。

練習3：得点を計算する

以下はグループで調整できる仮ルールです。まず全員の教材で同じ式を使います。

内訳	点数
脱出成功	300
残り時間	floor(max(0, time_left)) × 10
残りHP	max(0, hp) × 100
集めた鍵	重複しない鍵の数 × 50
ノーダメージ達成	200
残り60秒以上で脱出	100

失敗時は内訳の各点と合計を全て0点とし、ランキングへ登録しません。中途半端な失敗を繰り返す方が得にならないようにします。計算はfinishの中から呼ぶ専用関数へまとめます。

計算例A：成功、残り90秒、HP3、鍵3、被弾0回。300+900+300+150+200+100=1950点。

計算例B：成功、残り45秒、HP2、鍵3、被弾1回。300+450+200+150=1100点。

確認：59.9秒なら時間点は590、速度ミッションは未達。60.0秒なら600点と速度ボーナス100点。

練習4：結果を1回だけ確定する

finishは初回にResultを作ってself.resultに保存し、2回目以降は同じ結果を返します。終了画面が60FPSで描かれても、計算や保存を繰り返さないようにします。

```
def finish(self, *, won, time_left, hp, stage_id):
    if self.result is not None:
        return self.result
    # TODO: 内訳と達成状態からResultを作る
    return self.result
```

Gameではplayingからwon/lostへ変わる瞬間にfinishを呼びます。保存の一度だけ制御はGameのresult_savedで行い、draw_overlayはself.resultを読むだけにします。

練習5：ランキングを読み込む

保存先はゲームのルートを基準に saves/ranking.json とします。作業ディレクトリによって保存場所が変わらないよう、Path(__file__).resolve().parentを使います。

```
from pathlib import Path
import json

SAVE_PATH = Path(__file__).resolve().parent / 'saves' / 'ranking.json'
```

load_records(path)はlist[dict]を返します。ファイルなし、JSONの文法エラー、読み取り失敗なら空リストを返し、ゲームを止めません。

JSONとして読めても、トップがlistとは限りません。各行がdictであることを先に調べ、stage_idがstr、scoreが非負のint、time_leftが非負で有限の数、hpが0〜3のintかを確認し、不正な行を除外します。boolを数値として受け付けないようtypeを確認します。

辞書の追加項目は無視して構いません。読み込んだ値を使ってPythonコードを実行する処理は不要です。

練習6：ランキングを保存する

save_record(path, result)は成功ならTrue、保存できなければFalseを返します。成功結果のみ対象です。既存の有効な記録へ1件追加し、scoreの降順、同点ならtime_leftの降順で並べ、上位5件にします。

親フォルダをmkdir(parents=True, exist_ok=True)で用意します。一時ファイルへUTF-8のJSONを書いてからreplaceで置き換えると、書き込み途中の内容が本番ファイルへ残りにくくなります。

```
temporary = path.with_suffix('.tmp')
# TODO: recordsをJSON文字列にしてtemporaryへ書く
# TODO: temporary.replace(path)で確定する
```

OSErrorは呼び出し元へFalseで伝えます。画面には保存失敗を表示し、脱出成功そのものを取り消しません。saves/は.gitignoreへ追加し、個人の記録を共有コードに混ぜません。

練習7：結果画面へ表示する

画面に合計点、時間点、HP点、鍵点、ミッション2種類の成否を表示します。ランキングは上位5件の点数とステージ名を表示し、実装初期は名前入力を省きます。

Game.draw_overlayの結果分岐を拡張します。必要なら表示行を減らし、文字を画面外へ出さないようにします。保存済みランキングはプレイ終了時に読み込み直し、毎フレームファイルを開かないでください。

動作確認表

準備・操作	期待する結果
計算例A	合計1950点
計算例B	合計1100点
被弾後に回復してHP3	ノーダメージは未達
同じ鍵を2回通知	鍵点は1個分
finishを2回呼ぶ	結果が二重作成されない
保存ファイルなし	空ランキングで起動
壊れたJSON / 正しいJSONだが辞書	空ランキングで起動
6件以上の記録	上位5件だけ保存
保存できない場所を指定	Falseを返し、ゲームは継続
再挑戦	被弾数、取得鍵、resultを初期化

提出前の確認

テストでは一時ディレクトリを使い、本人の本当の記録を消さないようにします。例外を全部まとめて無視せず、JSONDecodeErrorやOSErrorなど想定した例外を扱います。

missions.py、結果画面の接続、README、.gitignoreのsaves/をコミットします。PRには得点式、境界値、保存形式、失敗時の挙動を記載します。

基準コード：https://github.com/kusupon/ProjExD_Group13/blob/33fe9905c183b321d1988d66041feb8b7e74998f/game.py

Python JSON：https://docs.python.org/3.13/library/json.html